```
at beginWork (react-dom_client.js?v=f7e0d6e5:7046:20)
at runWithFiberInDEV (react-dom_client.js?v=f7e0d6e5:724:18)
at performUnitOfWork (react-dom_client.js?v=f7e0d6e5:10829:98
at workLoopSync (react-dom_client.js?v=f7e0d6e5:10690:43)
at renderRootSync (react-dom_client.js?v=f7e0d6e5:10673:13)
at performWorkOnRoot (react-dom_client.js?v=f7e0d6e5:10321:46
at BuggyComponent (ErrorBoundaryExample.tsx:4:9)
at ErrorBoundaryExample (<anonymous>)
at App (<anonymous>)
```

Similar to `createRoot`, `hydrateRoot` also has three error hooks: `onCaughtError`, `onUncaughtError`, and `onRecoverableError`. By default, these hooks log errors to the console. However, they provide an opportunity for developers to implement more advanced error reporting mechanisms.

In an interview, it is valuable to highlight how these hooks fit into modern frontend error management strategies:

- *Centralized logging*—Discuss how these hooks can be wired to external monitoring services like Sentry, Datadog, or LogRocket to capture and analyze errors across the entire React tree.
- *Error categorization*—Show understanding of distinguishing between recoverable (e.g., hydration mismatches) and critical errors (e.g., component crashes) to prioritize error handling strategies.
- *Graceful degradation*—Explain how `onRecoverableError` can be used to maintain partial functionality in the UI while informing users about degraded experiences.
- *SSR considerations*—Point out that `hydrateRoot` is key for error handling during the hydration phase of server-rendered React apps and why catching mismatches early can prevent UI inconsistencies.

This knowledge not only demonstrates familiarity with React's latest error-handling APIs but also shows awareness of real-world practices for building resilient, observable systems.

A.7 React resources for Angular and Vue developers

React is the most widely used library for modern web applications. Although